LAB5 MCU3
실험 후 보고서

실험자 : 권진혁
학번 : 2023440007
학과 : 전자전기컴퓨공학부
실험 일시 : 2026. 11. 02.

1. 과제8 - MCU 송신 파형 확인

· 실험 목적

STM32 MCU를 SPI 마스터로 동작시켜 데이터를 송신하고, 오실로스코프를 이용하여 SCK, MOSI, CS 신호의 파형을 측정한다. 이를 통해 SPI Mode 0 통신의 동작 특성, 데이터의 비트 전송 순서(MSB First), 클록 주파수 및 신호 타이밍 관계를 확인하고 SPI 통신 원리를 이해한다. 또한, 송신 데이터를 0x55, 0xA5, 0x00, 0xFF 등으로 변경하면서 실제 MOSI 파형이 전송 데이터와 일치하는지 검증한다.

· 핵심 코드 및 설명

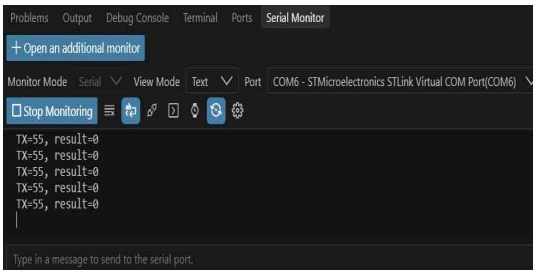
```c
void polling_routine(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms < 1000U) return;
    last_ms = now_ms;

    uint8_t tx = 0x55;
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef result =
        HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_SET);

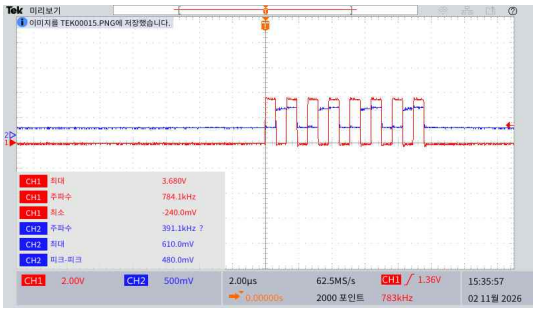
    printf("TX=%02X, result=%d\r\n",
           (unsigned int)tx, (int)result);
}
```

STM32의 SPI1을 이용하여 1초마다 1바이트(0x55)를 송신하는 기능을 수행한다. SPI 통신 시작 전 CS(Chip Select) 신호를 LOW로 설정하여 슬레이브 장치를 선택하고, SPI 송신이 완료되면 CS 신호를 HIGH로 복귀시켜 통신을 종료한다. 송신 결과는 UART를 통해 PC 시리얼 모니터에 출력된다.

· 실행 결과

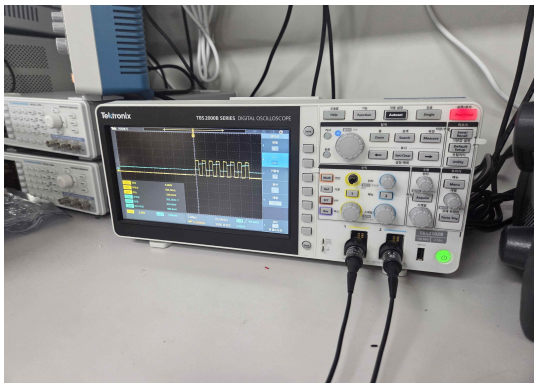


시리얼 모니터에서 TX=55, result=0이 1초 간격으로 반복적으로 출력되는 것을 확인하였다. 이는 STM32가 SPI를 통해 데이터 0x55를 정상적으로 송신하였음을 의미한다. 또한 반환값 result=0(HAL_OK)를 통해 SPI 전송이 오류 없이 완료되었음을 검증하였다.



오실로스코프를 이용하여 SPI 통신의 SCK와 MOSI 신호를 측정하였다. 측정 결과 SCK에서 약 784kHz의 클록이 생성되었으며, 1바이트 전송에 해당하는 8개의 클록 펄스를 확인하였다. MOSI 신호는 송신 데이터인 0x55(01010101_2)에 대응하는 패턴으로 출력되었으며, 각 비트가 클록에 동기되어 안정적으로 전송되는 것을 확인하였다. 또한, SPI Mode 0의 타이밍 특성과 일치하는 파형이 관찰되어 SPI 송신 기능이 정상적으로 구현되었음을 검증하였다.

- 오실로스코프



2. 과제9 - MCU → MCU 수신 확인

· 실험 목적

과제 9는 두 대의 MCU를 SPI 통신으로 연결하여 한쪽 MCU가 마스터(송신기), 다른 MCU가 슬레이브(수신기)로 동작하도록 구성하고, 송신한 데이터가 수신 측에서 정상적으로 수신되는지를 확인하는 실험이다. 수신된 데이터는 VCP UART를 통해 PC의 시리얼 모니터에 출력하여 송신 데이터와 비교하며, 마스터와 슬레이브 역할을 교대하여 SPI 송수신 동작을 모두 검증한다.

· 핵심 코드 및 설명

- 송신

```
void polling_routine(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms < 1000U) return;
    last_ms = now_ms;

    uint8_t tx = 0x55;
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef result =
        HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_SET);

    printf("TX=%02X, result=%d\r\n",
           (unsigned int)tx, (int)result);
}
```

STM32의 SPI1을 이용하여 1초마다 1바이트(0x55)를 송신하는 기능을 수행한다. SPI 통신 시작 전 CS(Chip Select) 신호를 LOW로 설정하여 슬레이브 장치를 선택하고, SPI 송신이 완료되면 CS 신호를 HIGH로 복귀시켜 통신을 종료한다. 송신 결과는 UART를 통해 PC 시리얼 모니터에 출력된다.

- 수신

```
#define SPI_NSS_RX_ece2_Pin       GPIO_PIN_4
#define SPI_NSS_RX_ece2_GPIO_Port GPIOA
```

SPI 슬레이브가 사용할 NSS(CS) 신호의 핀 정보를 정의하는 부분이다. NSS는 마스터가 통신 시작과 종료를 알리기 위해 사용하는 Chip Select 신호로, 본 실험에서는 PA4 핀을 NSS 입력으로 사용한다. 이후 초기화 코드와 수신 코드에서 해당 핀을 쉽게 참조할 수 있도록 매크로 형태로 선언하였으며, 마스터 MCU의 CS 신호가 이 핀으로 전달되어 슬레이브 MCU가 SPI 통신 시점을 인식할 수 있도록 한다.

```
void ece2_474_init(void)
{
    if (HAL_SPI_DeInit(&hspi1) != HAL_OK)
        Error_Handler();

    hspi1.Init.Mode = SPI_MODE_SLAVE;
    hspi1.Init.NSS = SPI_NSS_HARD_INPUT;
    if (HAL_SPI_Init(&hspi1) != HAL_OK)
        Error_Handler();

    GPIO_InitTypeDef gpio = {0};
    gpio.Pin = SPI_NSS_RX_ece2_Pin;
    gpio.Mode = GPIO_MODE_AF_PP;
    gpio.Pull = GPIO_PULLUP;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    gpio.Alternate = GPIO_AF5_SPI1;
    HAL_GPIO_Init(SPI_NSS_RX_ece2_GPIO_Port, &gpio);
}
```

SPI1을 슬레이브 모드로 설정하고 NSS 신호를 하드웨어 입력으로 사용하도록 구성하는 역할을 수행한다. SPI_MODE_SLAVE를 설정하여 외부 마스터가 생성한 SCK와 CS 신호에 동기되어 동작하게 하며, SPI_NSS_HARD_INPUT을 사용하여 NSS 신호를 자동으로 감지하도록 한다. 또한, PA4 핀을 SPI1의 NSS 기능을 수행하는 대

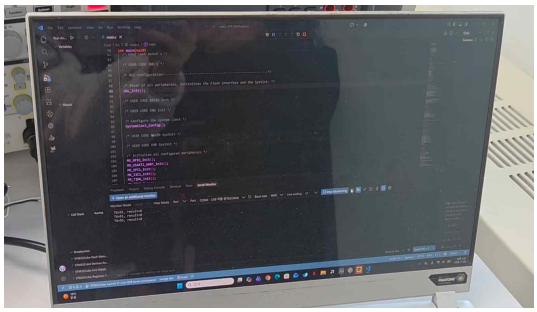
체 기능(Alternate Function) 핀으로 초기화하여 마스터 MCU의 CS 신호를 직접 수신할 수 있도록 구성한다. 이를 통해 슬레이브 MCU는 별도의 제어 없이 SPI 통신이 시작될 때 자동으로 수신을 준비할 수 있다.

```
void polling_routine(void)
{
    uint8_t rx = 0;
    HAL_StatusTypeDef result =
        HAL_SPI_Receive(&hspi1, &rx, 1U, 2000U);
    if (result == HAL_OK)
    {
        printf("RX=%02X\r\n", (unsigned int)rx);
    }
    else if (result == HAL_TIMEOUT)
    {
        printf("RX timeout: check sender, SCK, MOSI, CS and GND\r\n");
    }
    else
    {
        printf("RX error=%d\r\n", (int)result);
    }
}
```

SPI 통신을 통해 데이터를 실제로 수신하는 역할을 담당한다. HAL_SPI_Receive() 함수를 이용하여 SPI1에서 1바이트 데이터를 수신하고, 수신이 성공하면 UART를 통해 수신된 값을 시리얼 모니터에 출력한다. 송신 측에서 0x55를 전송하면 수신 측에는 RX=55가 출력되어 데이터가 정상적으로 전달되었음을 확인할 수 있다. 또한, 2초 동안 데이터가 들어오지 않을 경우 타임아웃 메시지를 출력하여 배선 오류나 통신 문제를 확인할 수 있도록 하였으며, 그 밖의 통신 오류가 발생하면 오류 코드를 출력하여 디버깅이 가능하도록 구현하였다.

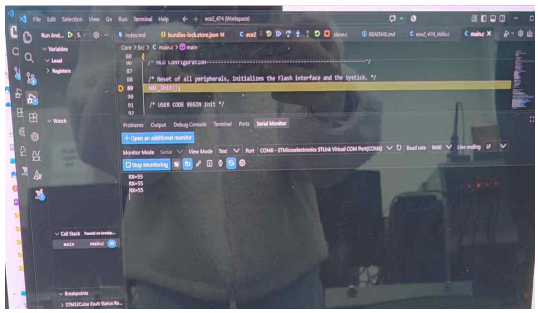
· **실행 결과**

- 송신 측 화면



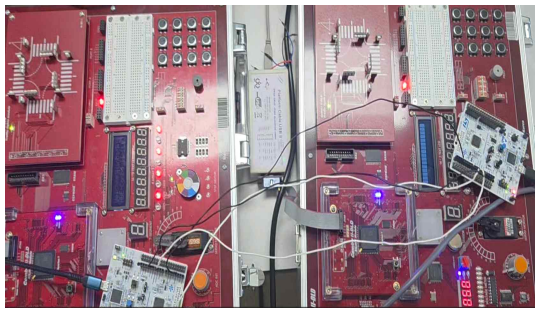
송신 측 MCU를 실행한 결과 시리얼 모니터에 TX=55, result=0이 반복적으로 출력되는 것을 확인하였다. 이는 SPI 마스터가 0x55 데이터를 주기적으로 송신하고 있으며, 반환값 0(HAL_OK)을 통해 전송 과정에서 오류가 발생하지 않았음을 의미한다. 따라서 송신 MCU의 SPI 전송 기능이 정상적으로 동작함을 확인하였다.

- 수신 측 화면



수신 측 MCU를 실행한 결과 시리얼 모니터에 RX=55가 반복적으로 출력되는 것을 확인하였다. 이는 송신 MCU가 SPI를 통해 전송한 데이터 0x55가 수신 MCU에 정상적으로 전달되었음을 의미한다. 또한 송신 측에서 출력된 TX=55와 수신 측의 RX=55가 일치하여 데이터 손실이나 오류 없이 SPI 통신이 수행되었음을 확인할 수 있었다. 따라서 두 MCU 간 SPI 마스터-슬레이브 통신이 정상적으로 구현되었으며, 수신 기능 또한 정상적으로 동작함을 검증하였다.

- FPGA



3. 과제10 - MCU → FPGA 수신 구현

· 실험 목적

MCU를 SPI 마스터로 사용하고 FPGA를 SPI 슬레이브로 구성하여, MCU가 전송한 데이터를 FPGA가 정상적으로 수신하도록 구현하는 실험이다. FPGA에서 CS 신호 감지, SCK 상승 에지 검출, 8비트 데이터 수신 및 수신 완료 처리를 Verilog로 구현하고, 수신한 데이터를 FPGA의 UART를 통해 PC에 출력하여 MCU의 송신값과 비교한다. 이를 통해 SPI 통신 프로토콜의 수신 동작 원리와 FPGA 기반 SPI 수신기 설계 방법을 이해한다.

· 핵심 코드 및 설명

```
src > edge_detector.v > {} edge_detector > always > {} if
1  module edge_detector (
2      input clk,
3      input rst_n,
4      input sig_in,
5
6      output rise_tick
7  );
8
9  reg sig_d;
10
11 always @(posedge clk or negedge rst_n)
12 begin
13     if(!rst_n)
14         sig_d <= 1'b0;
15     else
16         sig_d <= sig_in;
17 end
18
19 assign rise_tick = (~sig_d) & sig_in;
20
21 endmodule
```

edge_detector.v는 SPI 클록(SCK)의 상승 에지를 검출하기 위한 모듈이다. FPGA 내부 시스템 클록을 기준으로 이전 SCK 상태를 저장한 뒤 현재 상태와 비교하여 LOW에서 HIGH로 변하는 순간에만 rise_tick 신호를 발생시킨다. SPI Mode 0에서는 상승 에지에서 데이터를 샘플링하므로, 본 모듈은 SPI 수신기가 MOSI 데이터를 정확한 시점에 읽을 수 있도록 기준 신호를 제공하는 역할을 수행한다.

```
src > spi_rx.v > {} spi_rx > always > {} if
1  module spi_rx (
2      input clk,
3      input rst_n,
4
5      input cs_n,
6      input sck_rise,
7      input mosi,
8
9      output reg [7:0] rx_data,
10     output reg rx_done
11 );
12
13 reg [7:0] shift_reg;
14 reg [2:0] bit_cnt;
15
16 always @(posedge clk or negedge rst_n)
17 begin
18     if(!rst_n)
19     begin
20         shift_reg <= 8'd0;
21         bit_cnt   <= 3'd0;
22         rx_data   <= 8'd0;
23         rx_done   <= 1'b0;
24     end
25     else
26     begin
27         rx_done <= 1'b0;
28
29         if(cs_n)
30         begin
31             bit_cnt <= 3'd0;
32         end
33         else
34         begin
35             if(sck_rise)
36             begin
37                 shift_reg <= {shift_reg[6:0], mosi};
38
39                 if(bit_cnt == 3'd7)
40                 begin
41                     rx_data <= {shift_reg[6:0], mosi};
42                     rx_done <= 1'b1;
43                     bit_cnt <= 3'd0;
44                 end
45                 else
46                 begin
47                     bit_cnt <= bit_cnt + 1'b1;
48                 end
49             end
50         end
51     end
52 end
53
54 endmodule
```

spi_rx.v는 FPGA의 SPI 슬레이브 수신 기능을 담당하는 핵심 모듈이다. CS 신호가 LOW 상태일 때만 통신을 활성화하며, SCK 상승 에지가 검출될 때마다 MOSI 데이터를 시프트 레지스터에 순차적으로 저장한다. 비트 카운터를 이용하여 총 8비트가 수신되었는지 확인하고, 수신이 완료되면 최종 데이터를 rx_data에 저장한 후 rx_done 신호를 1클록 동안 발생시킨다. 이를 통해 MCU가 송신한 1바이트 데이터를 FPGA가 정상적으로 수신할 수 있도록 구현하였다.

```
src > top_spi_rx.v > { } top_spi_rx > { } spi_rx > { } U_SPI_RX
1  module top_spi_rx(
2
3      input clk,
4      input rst_n,
5
6      input spi_cs_n,
7      input spi_sck,
8      input spi_mosi,
9
10     output [7:0] rx_data,
11     output rx_done
12 );
13
14 wire sck_rise;
15
16 edge_detector U_EDGE(
17     .clk(clk),
18     .rst_n(rst_n),
19     .sig_in(spi_sck),
20     .rise_tick(sck_rise)
21 );
22
23 spi_rx U_SPI_RX(
24     .clk(clk),
25     .rst_n(rst_n),
26
27     .cs_n(spi_cs_n),
28     .sck_rise(sck_rise),
29     .mosi(spi_mosi),
30
31     .rx_data(rx_data),
32     .rx_done(rx_done)
33 );
34
35 endmodule
```

top_spi_rx.v는 SPI 수신 시스템의 최상위 모듈로서 각 하위 모듈을 연결하는 역할을 수행한다. SPI 클록 신호는 edge_detector 모듈로 전달되어 상승 에지 신호를 생성하고, 생성된 에지 신호와 MOSI 및 CS 신호는 spi_rx 모듈로 입력된다. 최상위 모듈은 SPI 수신기 전체를 하나의 블록처럼 사용할 수 있도록 구성하며, 최종적으로 수신 데이터(rx_data)와 수신 완료 신호(rx_done)를 외부로 출력한다.

```
src > top_spi_rx.v > { } top_spi_rx
1  module top_spi_rx(
2
3      input clk,
4      input rst_n,
5
6      input spi_cs_n,
7      input spi_sck,
8      input spi_mosi,
9
10     output [7:0] rx_data,
11     output rx_done
12 );
13
14 wire sck_rise;
15
16 edge_detector U_EDGE(
17     .clk(clk),
18     .rst_n(rst_n),
19     .sig_in(spi_sck),
20     .rise_tick(sck_rise)
21 );
22
23 spi_rx U_SPI_RX(
24     .clk(clk),
25     .rst_n(rst_n),
26
27     .cs_n(spi_cs_n),
28     .sck_rise(sck_rise),
29     .mosi(spi_mosi),
30
31     .rx_data(rx_data),
32     .rx_done(rx_done)
33 );
34
35 endmodule
```

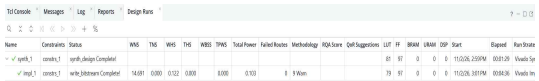
tb_top_spi_rx.v는 SPI 수신기를 검증하기 위한 TB이다. MCU 대신 가상의 SPI 마스터 역할을 수행하여 CS, SCK, MOSI 신호를 생성하고 0x55와 0xA5 데이터를 순차적으로 전송한다. 또한, 시뮬레이션 과정의 모든 주요 신호를 spi_rx.vcd 파일로 저장함으로써 파형을 분석할 수 있도록 하였다. 이를 통해 CS 활성화, 클록 상승 에지 검출, 8비트 데이터 수신, 수신 완료 신호 발생 과정을 시각적으로 검증할 수 있다.

· 실행 결과

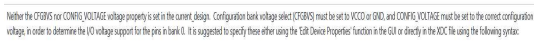
- Bit Hash

```
Algorithm : SHA256
Hash      : 85A80DF27833C8CCC16F554407CD97E65C92C55FC36828BBEACCA40BF1F8141A4
Path      : D:\Engineering\Vivado_Work\Lab7\lab7_spi_rx\project_1.runs\impl_1\lab7_spi_rx_uart.bit
```

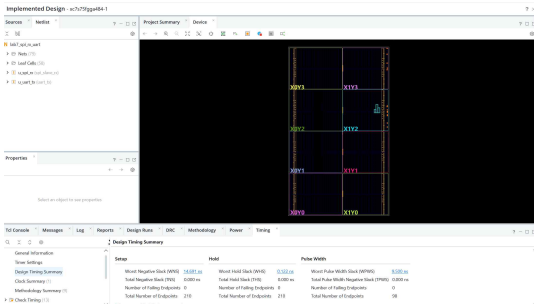
- Bitstream



- DRC



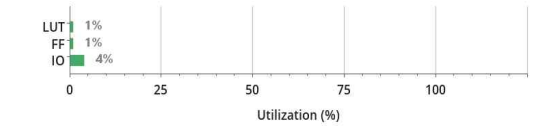
- Timing



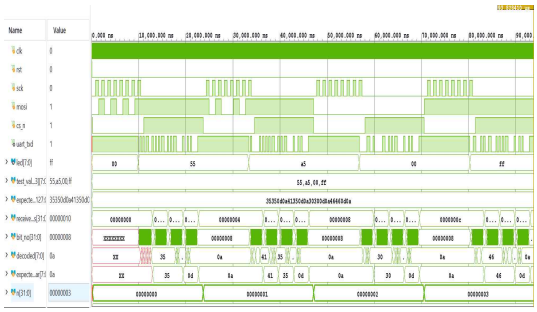
- Utilization

Summary

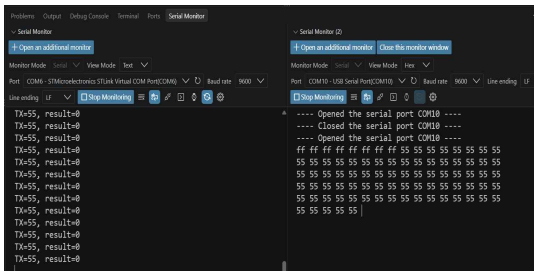
Resource	Utilization	Available	Utilization %
LUT	79	48000	0.16
FF	97	96000	0.10
IO	14	338	4.14



- Simulation

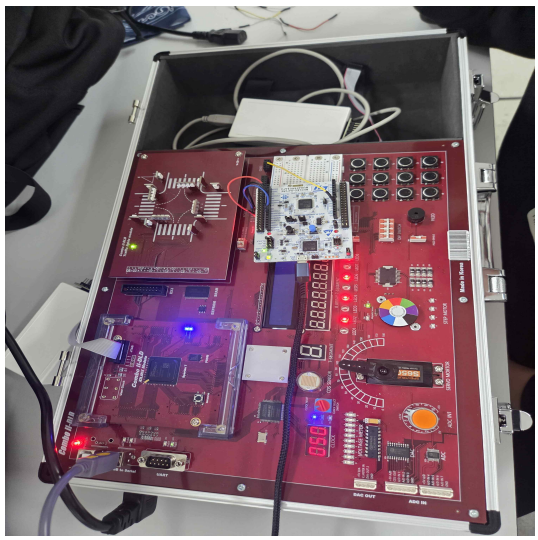


- Serial Monitor



송신 측 MCU의 시리얼 모니터에서 TX=55, result=0이 반복적으로 출력되어 SPI 송신 기능이 정상적으로 동작함을 확인하였다. 또한, FPGA 수신 측 UART 출력에서는 초기 일부 FF 데이터 이후 55가 지속적으로 출력되었으며, 이는 MCU가 SPI를 통해 송신한 0x55 데이터를 FPGA가 정상적으로 수신하였음을 의미한다. FPGA는 CS 신호 검출, SCK 상승 에지 검출, 8비트 데이터 수신 및 UART 출력 기능을 정상적으로 수행하였으며, 이를 통해 MCU와 FPGA 간 SPI 통신이 성공적으로 구현되었음을 검증하였다.

- FPGA



4. 과제11 - FPGA → MCU 송신 구현

· 실험 목적

본 실험의 목적은 FPGA를 SPI 마스터로 동작시켜 데이터를 송신하고 MCU를 SPI 슬레이브로 동작시켜 데이터를 수신하는 것이다. FPGA에서 CS, SCK, MOSI 신호를 생

성하여 SPI 송신 기능을 구현하고, MCU의 UART 출력 결과를 이용하여 수신 데이터의 정확성을 확인한다. 이를 통해 SPI 통신 프로토콜의 송신 과정과 FPGA 기반 SPI 송신기 설계 방법을 이해하고, FPGA와 MCU 간 데이터 통신이 정상적으로 이루어지는지 검증한다.

· 핵심 코드 및 설명

```
src > spi_tx.v > spi_tx > always > if > else > case
module spi_tx
#(
    parameter CLK_DIV = 50
)
(
    input clk,
    input rst_n,

    input start,
    input [7:0] tx_data,

    output reg spi_cs_n,
    output reg spi_sck,
    output reg spi_mosi,

    output reg tx_done
);

reg [7:0] shift_reg;
reg [2:0] bit_cnt;

reg [15:0] clk_cnt;

reg [2:0] state;

localparam IDLE  = 3'd0;
localparam LOAD  = 3'd1;
localparam SHIFT = 3'd2;
localparam DONE  = 3'd3;

always @(posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        state <= IDLE;

        spi_cs_n <= 1'b1;
        spi_sck  <= 1'b0;
        spi_mosi <= 1'b0;

        tx_done  <= 1'b0;

        shift_reg <= 8'd0;
        bit_cnt   <= 3'd0;
        clk_cnt   <= 16'd0;
    end
    else
    begin
        tx_done <= 1'b0;

        case(state)

        IDLE:
        begin
            spi_cs_n <= 1'b1;
            spi_sck  <= 1'b0;
            if(start)
                state <= LOAD;
        end

        LOAD:
        begin
            spi_cs_n <= 1'b0;

            shift_reg <= tx_data;
            bit_cnt   <= 3'd7;

            spi_mosi <= tx_data[7];

            clk_cnt <= 16'd0;

            state <= SHIFT;
        end

        SHIFT:
        begin
            clk_cnt <= clk_cnt + 1'b1;

            if(clk_cnt == CLK_DIV-1)
            begin
                clk_cnt <= 0;

                spi_sck <= ~spi_sck;

                if(spi_sck)
                begin
                    if(bit_cnt == 0)
                    begin
                        state <= DONE;
                    end
                    else
                    begin
                        shift_reg <= {shift_reg[6:0],1'b0};
                        bit_cnt <= bit_cnt - 1'b1;

                        spi_mosi <= shift_reg[6];
                    end
                end
            end
        end

        DONE:
        begin
            spi_cs_n <= 1'b1;
            spi_sck  <= 1'b0;

            tx_done <= 1'b1;

            state <= IDLE;
        end

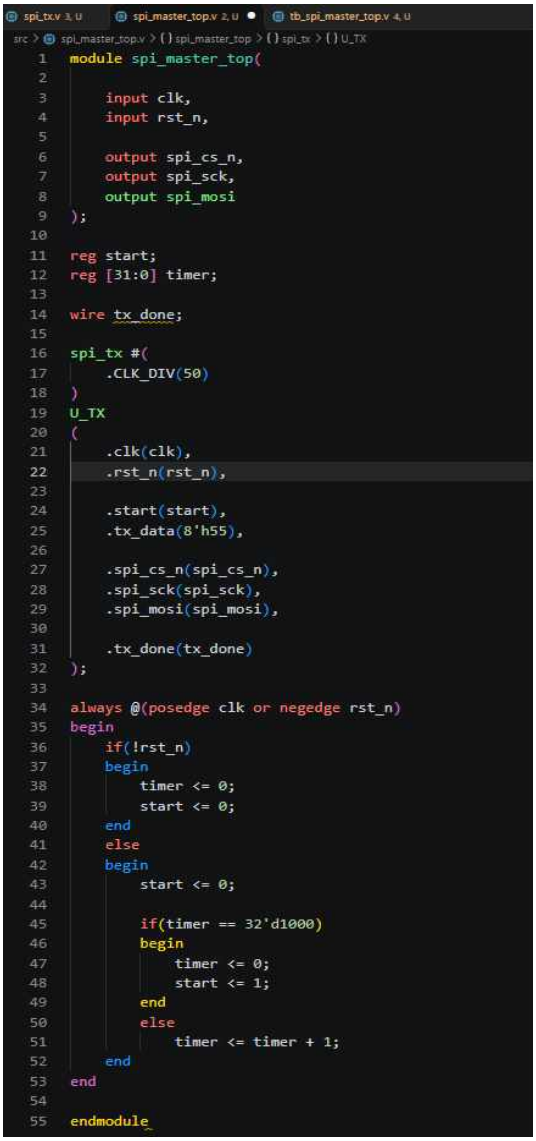
        endcase
    end
end

endmodule
```

spi_tx.v는 FPGA에서 SPI 마스터 송신 기능을 수행하는 핵심 모듈이다. 송신 시작 신호(start)가 입력되면 전송할 데이터(tx_data)를 시프트 레지스터에 저장하고 CS 신호를 LOW로 활성화한다. 이후 내부 클록 분주기를 이용하여 SPI 클록(SCK)을 생성하며, MOSI 라인을 통해 데이터를 상위 비트(MSB)부터 순차적으로 전송한다. 모든 8비트 전송이 완료되면 CS 신호를 HIGH로

복귀시키고 tx_done 신호를 발생시켜 송신 완료 상태를 알린다. 이를 통해 FPGA가 SPI 마스터로서 MCU에 데이터를 전송할 수 있도록 구현하였다.

MCU와 통신할 수 있도록 구성하였다. 이를 통해 별도의 제어 없이 FPGA가 주기적으로 데이터를 송신하는 SPI 마스터 동작을 수행할 수 있다.



```
module spi_master_top(

    input clk,
    input rst_n,

    output spi_cs_n,
    output spi_sck,
    output spi_mosi
);

reg start;
reg [31:0] timer;

wire tx_done;

spi_tx #(
    .CLK_DIV(50)
)
U_TX
(
    .clk(clk),
    .rst_n(rst_n),

    .start(start),
    .tx_data(8'h55),

    .spi_cs_n(spi_cs_n),
    .spi_sck(spi_sck),
    .spi_mosi(spi_mosi),

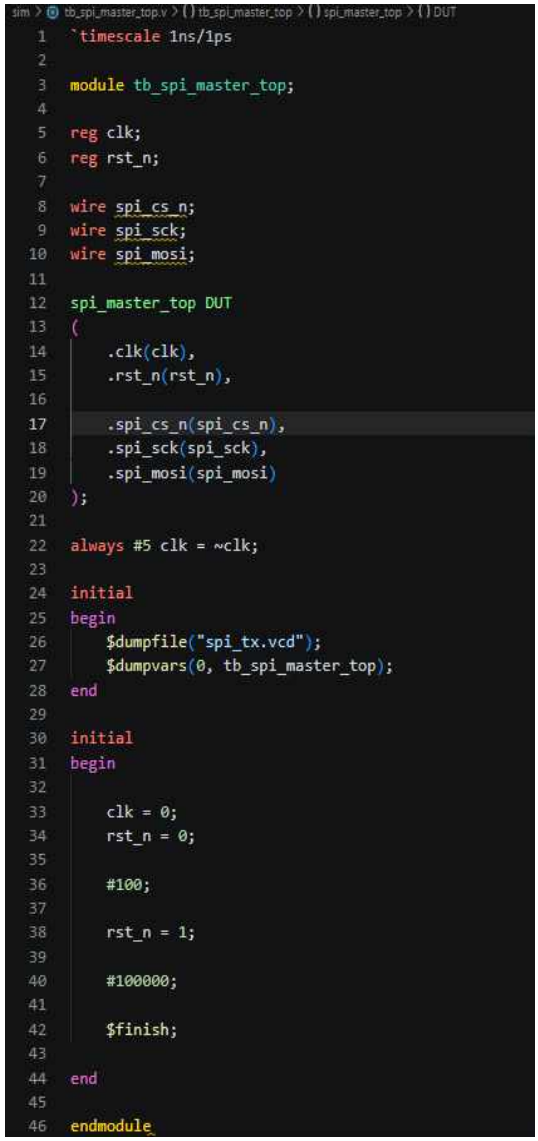
    .tx_done(tx_done)
);

always @(posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        timer <= 0;
        start <= 0;
    end
    else
    begin
        start <= 0;

        if(timer == 32'd1000)
        begin
            timer <= 0;
            start <= 1;
        end
        else
            timer <= timer + 1;
    end
end

endmodule
```

spi_master_top.v는 SPI 송신 시스템의 최상위 모듈로서 송신 제어 기능을 담당한다. 내부 타이머를 이용하여 일정 주기마다 송신 시작 신호를 생성하고, 송신 데이터로 0x55 값을 설정하여 spi_tx 모듈에 전달한다. 또한, SPI 송신 모듈의 출력인 CS, SCK, MOSI 신호를 외부 핀으로 연결하여 실제



```
`timescale 1ns/1ps

module tb_spi_master_top;

reg clk;
reg rst_n;

wire spi_cs_n;
wire spi_sck;
wire spi_mosi;

spi_master_top DUT
(
    .clk(clk),
    .rst_n(rst_n),

    .spi_cs_n(spi_cs_n),
    .spi_sck(spi_sck),
    .spi_mosi(spi_mosi)
);

always #5 clk = ~clk;

initial
begin
    $dumpfile("spi_tx.vcd");
    $dumpvars(0, tb_spi_master_top);
end

initial
begin

    clk = 0;
    rst_n = 0;

    #100;

    rst_n = 1;

    #100000;

    $finish;

end

endmodule
```

tb_spi_master_top.v는 SPI 송신 회로의 기능을 검증하기 위한 TB이다. FPGA 내부 클록과 리셋 신호를 생성하고, 최상위 모듈(spi_master_top)을 대상으로 시뮬레이션을 수행한다. 리셋 해제 후 SPI 송신기가 자동으로 동작하도록 하여 CS 신호 활성화, 클록 생성, MOSI 데이터 전송 및 송신 완료

신호 발생 과정을 검증할 수 있도록 구성하였다. 이를 통해 실제 FPGA에 다운로드하기 전에 설계가 정상적으로 동작하는지 확인할 수 있다.

· 실행 결과

- Bit Hash



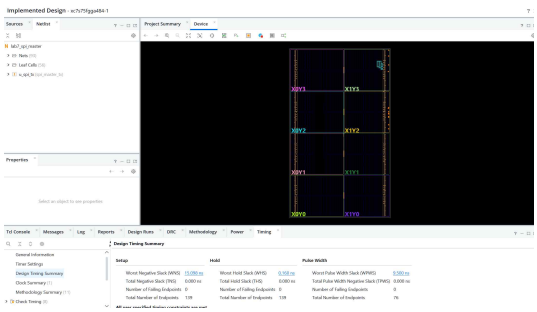
- Bitstream



- DRC



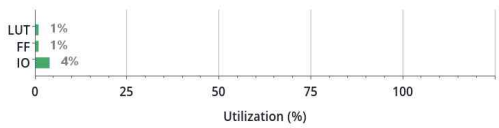
- Timing



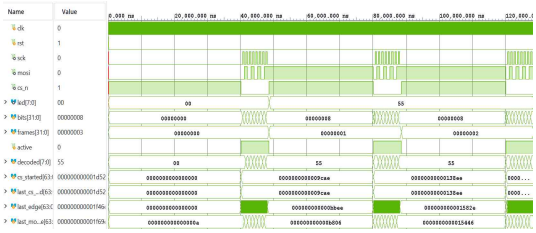
- Utilization

Summary

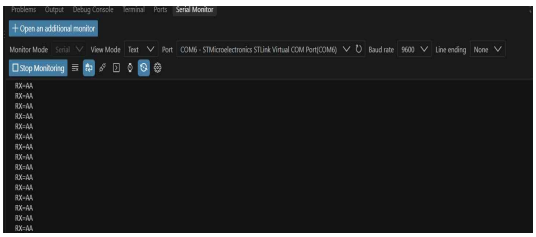
Resource	Utilization	Available	Utilization %
LUT	48	48000	0.10
FF	75	96000	0.08
IO	13	338	3.85



- Simulation

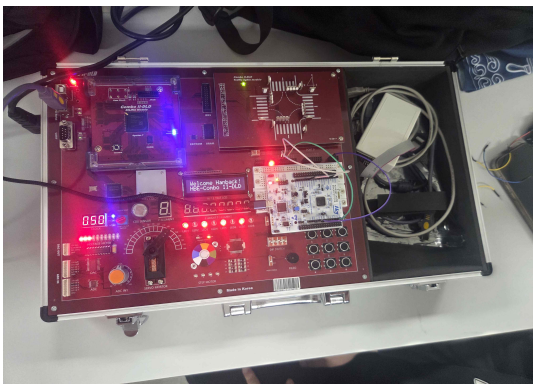


- Serial Monitor



FPGA에 구현한 SPI 마스터 송신기를 실행한 결과, MCU의 시리얼 모니터에서 RX=AA가 반복적으로 출력되는 것을 확인하였다. 이는 FPGA가 SPI를 통해 0xAA 데이터를 정상적으로 송신하였으며, MCU가 이를 오류 없이 수신하였음을 의미한다. 또한, FPGA 보드와 MCU가 정상적으로 연결된 상태에서 데이터가 안정적으로 반복 수신되어 CS, SCK, MOSI 신호가 올바르게 동작함을 확인하였다. 따라서 FPGA 기반 SPI 송신기와 MCU 기반 SPI 수신기가 정상적으로 구현되었으며, FPGA와 MCU 간 SPI 통신이 성공적으로 수행되었음을 검증하였다.

- FPGA



5. 과제12 - FPGA → FPGA 송수신 구현

· 실험 목적

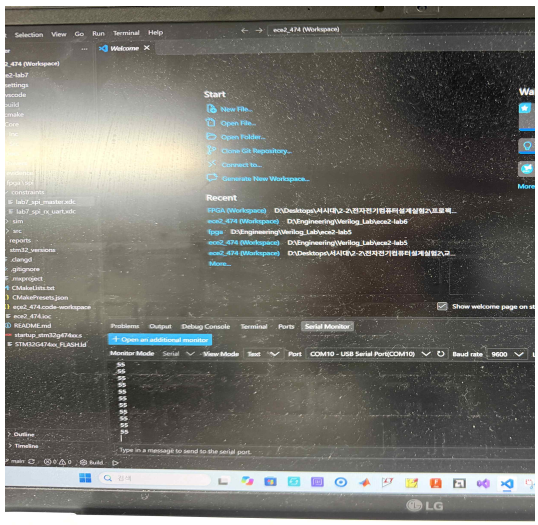
본 실험의 목적은 두 대의 FPGA를 SPI 통신으로 연결하여 FPGA 간 데이터 송수신 시스템을 구현하는 것이다. 송신 FPGA에서는 SPI 마스터를 구현하여 데이터를 전송하고, 수신 FPGA에서는 SPI 슬레이브를 구현하여 데이터를 수신한다. 또한 수신된 데이터를 UART 또는 디버그 출력 장치를 통해 확인함으로써 송신 데이터와 수신 데이터의 일치 여부를 검증하고, FPGA 기반 SPI 송수신 회로의 동작 원리와 설계 방법을 이해한다.

· 핵심 코드 및 설명

과제 12에서는 이전 과제에서 구현한 FPGA 기반 SPI 송신기와 SPI 수신기를 통합하여 FPGA 간 SPI 통신 시스템을 구성하였다. 송신 FPGA는 SPI 마스터로 동작하며 데이터를 전송하고, 수신 FPGA는 SPI 슬레이브로 동작하며 데이터를 수신하도록 설계하였다. 송신 데이터와 수신 데이터를 비교한 결과 동일한 값이 확인되어 FPGA 간 SPI 통신이 정상적으로 수행됨을 검증하였다. 또한, 과제 10과 과제 11에서 검증한 모듈을 재사용함으로써 SPI 송수신 회로의 통합 동작을 확인할 수 있었다.

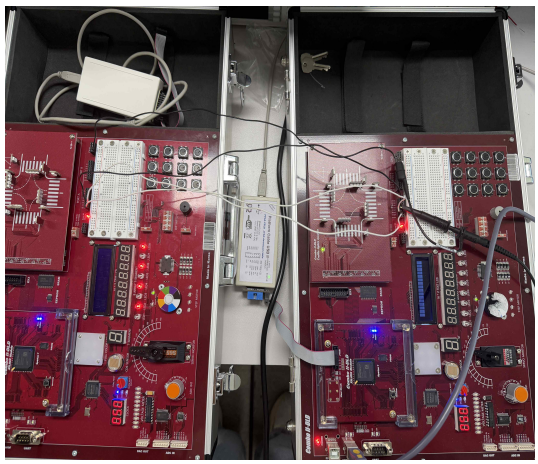
· 실험 결과

- Serial Monitor



실험 결과 수신 FPGA의 시리얼 모니터에서 55가 반복적으로 출력되는 것을 확인하였다. 이는 송신 FPGA가 SPI를 통해 전송한 0x55 데이터가 수신 FPGA에 정상적으로 전달되었음을 의미한다. 또한 반복적인 송수신 과정에서도 동일한 결과가 지속적으로 출력되어 데이터 손실이나 통신 오류가 발생하지 않았음을 확인하였다. 따라서 송신 FPGA의 SPI 송신기와 수신 FPGA의 SPI 수신기가 정상적으로 동작하였으며, FPGA 간 SPI 통신이 성공적으로 구현되었음을 검증하였다.

- FPGA



6. 최종 고찰

본 실험에서는 SPI(Serial Peripheral Interface) 통신을 이용하여 MCU와 FPGA 간의 데이터 송수신 과정을 단계적으로 구현하고 검증하였다. 과제 8에서는 STM32 MCU를 SPI 마스터로 설정하여 데이터를 송신하고 오실로스코프를 통해 SCK, MOSI, CS 신호를 관찰함으로써 SPI 통신의 기본 원리와 신호 특성을 이해하였다. 이를 통해 SPI 통신이 클록 신호를 기준으로 동기적으로 데이터를 전송한다는 점과 데이터 패턴에 따른 MOSI 파형 변화를 확인할 수 있었다.

과제 9에서는 두 대의 MCU를 이용하여 SPI 송수신 시스템을 구축하였다. 송신 측의 데이터가 수신 측에 정상적으로 전달되는 것을 UART 출력으로 확인함으로써 SPI 마스터와 슬레이브 간 데이터 전송 과정을 검증하였다. 이를 통해 SPI 통신에서 배선 연결과 신호 타이밍의 중요성을 이해할 수 있었으며, 데이터가 손실 없이 전송되는 것을 확인하였다.

과제 10과 과제 11에서는 FPGA를 활용하여 SPI 수신기와 송신기를 각각 구현하였다. Verilog HDL을 이용하여 CS 신호 검출, SCK 상승 에지 검출, 시프트 레지스터, 비트 카운터, 상태 머신(FSM) 등을 설계하였으며, MCU와 FPGA 간 통신을 통해 구현된 회로가 정상적으로 동작함을 검증하였다. 특히 FPGA 내부에서 SPI 프로토콜을 직접 구현함으로써 하드웨어 설계 관점에서 데이터 송수신 과정을 이해할 수 있었고, 디지털 시스템 설계 능력을 향상시킬 수 있었다.

마지막으로 과제 12에서는 앞선 과제에서 구현한 SPI 송신기와 수신기를 통합하여 FPGA 간 SPI 통신을 구현하였다. 송신 FPGA의 데이터가 수신 FPGA에 정상적으로 전달되는 것을 UART 출력 결과를 통해 확인하였으며, 이전 과제에서 설계한 모듈을 재사용하여 보다 큰 시스템으로 확장할 수 있음을 확인하였다. 이를 통해 모듈화 설계의 중요성과 검증된 회로의 재사용성을 이해할 수 있었다.

이번 실험을 수행하면서 SPI 통신의 동작 원리뿐만 아니라 MCU와 FPGA의 역할 차이, 하드웨어 기술언어를 이용한 디지털 회로 설계 방법, 그리고 오실로스코프 및 시리얼 모니터를 활용한 검증 기법을 함께 익힐 수 있었다. 특히 단순히 제공된 IP를 사용하는 것이 아니라 SPI 프로토콜을 직접 구현하고 디버깅하는 과정을 통해 통신 시스템 설계 및 검증 능력을 향상시킬 수 있었으며, 향후 FPGA 기반 임베디드 시스템이나 SoC 설계 시 필요한 실무적 역량을 기를 수 있는 의미 있는 실험이었다.

7. 출처

[1] 서울시립대학교 전자전기컴퓨터공학부, STM32_3 · SPI로 MCU와 FPGA 연결, ECE2 디지털설계실험 교안

[2] STMicroelectronics, STM32F4 Series Reference Manual & HAL Driver Documentation, SPI Peripheral Documentation.

8. Git-Hub 및 Commit

· Git-Hub

· 6f30bf9